
Agentic Harness for Real-World Compilers

Yingwei Zheng^{*1} Cong Li^{*2} Shaohua Li³ Yuqun Zhang¹ Zhendong Su²

Abstract

Compilers are critical to modern computing, yet fixing compiler bugs is difficult. While recent large language model (LLM) advancements enable automated bug repair, compiler bugs pose unique challenges due to their complexity, deep cross-domain expertise requirements, and sparse, non-descriptive bug reports, necessitating compiler-specific tools. To bridge the gap, we introduce `llvm-autofix`, the first agentic harness designed to assist LLM agents in understanding and fixing compiler bugs. Our focus is on LLVM, one of the most widely used compiler infrastructures. Central to `llvm-autofix` are agent-friendly LLVM tools, a benchmark `llvm-bench` of reproducible LLVM bugs, and a tailored minimal agent `llvm-autofix-mini` for fixing LLVM bugs. Our evaluation demonstrates a performance decline of 60% in frontier models when tackling compiler bugs compared with common software bugs. Our minimal agent `llvm-autofix-mini` also outperforms the state-of-the-art by approximately 22%. This emphasizes the necessity for specialized harnesses like ours to close the loop between LLMs and compiler engineering. We believe this work establishes a foundation for advancing LLM capabilities in complex systems like compilers. GitHub: <https://github.com/dtcxzyw/llvm-autofix>

1. Introduction

Compilers serve as the foundational infrastructure for modern computing. They translate source code into efficient machine instructions. Virtually every piece of software relies on them for correct compilation and performance optimization, ranging from low-level OS kernels, machine learning (ML) frameworks, to high-level web applications

and ML models. Therefore, it is critical to repair and maintain compilers as soon as any issues arise. Recent advances in large language models (LLMs) present great potential in this direction (Yang et al., 2024b; Mathai et al., 2025).

However, unlike common software, compilers represent a unique category of large-scale, complex software systems. On one hand, compiler bugs are challenging to address. Common software bugs, such as Figure 1a, typically include natural language descriptions to aid developers’ understanding. Nevertheless, such descriptive information is absent in compiler bugs, complicating diagnosis and repair:

- Compiler *crash* bugs, such as Figure 1b, come with a test case `@test()`, also known as a *reproducer* in compiler engineering, and a stack trace to pinpoint where the compiler crashes while compiling the reproducer.
- Compiler *miscompilation* bugs, such as Figure 1c, offer a reproducer `@src()` and occasionally an input-output pair (known as a *counterexample*) to illustrate how the compiler incorrectly compiles the reproducer into faulty code `@tgt()`, leading to wrong output given the input.
- Other compiler bugs are even more intricate, basically presenting only a reproducer: *Slow compilations* and *compiler hangs* indicate compilers unexpectedly take a long time or forever to compile a reproducer; *Missed optimizations* denote compilers failing to apply certain beneficial code transformations on a reproducer.

On the other hand, understanding compilers requires specialized tooling and background rarely used in general software development, including lexing/parsing, type systems, IR design/optimization, and code generation, which typically take years for human engineers to master. While existing platforms such as SWE-bench (Jimenez et al., 2024) and SWE-agent (Yang et al., 2024b) effectively connect LLMs to standard bash tools for general software engineering tasks, they exhibit limited efficacy in compiler engineering.

The `llvm-autofix` harness. To close the loop between LLMs and compiler engineering, we introduce `llvm-autofix`, the first off-the-shelf harness designed to assist agents in understanding and fixing compiler bugs.

We focus on LLVM, one of the most widely used compilers

^{*}Equal contribution ¹Southern University of Science and Technology ²ETH Zurich ³The Chinese University of Hong Kong. Correspondence to: Shaohua Li <shaohuali@cuhk.edu.hk>, Yuqun Zhang <zhangyq@sustech.edu.cn>.

<pre>django.utils.http.parse_http_date two digit year check is incorrect Description: (last modified by Ad Timmering) RFC 850 does not mention this, but in RFC 7231 (and there's something similar in RFC 2822), there's the following quote: Recipients of a timestamp value in ... Current logic is hard coded to consider 0-69 to be in 2000-2069, and 70-99 to be 1970-1999, instead of comparing versus the current year.</pre>	<pre>;; opt -mtriple=s390x-linux-gnu -mcpu=z16 -O3 ... define i1 @test(i64 %0, i64 %1, ptr %2) { %gep44 = getelementptr i8, ptr null, i64 %0 %gep45 = getelementptr i8, ptr null, i64 %1 %4 = icmp ult ptr %gep44, %gep45 ... %b196 = icmp ult ptr %xmin50, %2 %res = and i1 %b095, %b196 ret i1 %res } ;; Assertion: isIntOrIntVectorTy() failed. ;; Original type expected to be a vector ;; of integers or a scalar integer. ;; Stack trace (#14--#1 omitted): ;; #15 slpvectorizer::BoUpSLP::getEntryCost ;; #14 ...</pre>	<pre>;; opt -passes=instsimplify -S ... define half @src(half noundef %mul) { %abs = call half @llvm.fabs.f16(half %mul) ... %abs2 = call half @llvm.fabs.f16(half %maxd) ret half %abs2 } ;; Incorrectly optimized into => define half @tgt(half noundef %mul) { %abs = call half @llvm.fabs.f16(half %mul) ... %maxd = select i1 %cmp1, half %mul, half 0xH0000 ret half %maxd } ;; Counterexample input: %x = #x8001 ;; Source @src output: #x0001 ;; Target @tgt output: #x8001</pre>
(a) Django #11848	(b) LLVM #99899	(c) LLVM #115824

Figure 1. **Compiler issues are challenging to diagnose and repair, in the absence of descriptive information.** This is a comparison between LLVM issues (crash and miscompilation) and common software issues from Django; all three issues are simplified for brevity.

powering languages from C/C++ and Rust to modern AI languages like Triton and Mojo. At present, we target its middle-end, which transforms an LLVM IR program into an optimized LLVM IR program via various optimization components. This target is due to the expressive, well-defined LLVM IR (LLVM, 2025b) and the complexity of its numerous, target-independent optimizations (LLVM, 2025c), which makes it the most prioritized end (Sun et al., 2016).

llvm-autofix is divided into three essential parts:

- A set of compiler-specific tools for building, reproducing, exploring, debugging, and testing LLVM. These tools provide an agent-friendly interface, eliminating unnecessary technical details, allowing agents to focus on the core aspects of bug localization and repair.
- A benchmark called llvm-bench consisting of 334 reproducible LLVM middle-end bugs, each accompanied by around 1.4 reproducers and 722 robust regression tests. They are separated into three splits based on the difficulty level: easy (76.3%), medium (13.2%), and hard (10.5%). Currently, the benchmark focuses on the two most common types of compiler bugs: crashes and miscompilations, as performance issues are often less prioritized by LLVM developers. We also maintain llvm-bench live, a continuously updated llvm-bench subset including only issues from the past year.
- A minimal agent called llvm-autofix-mini tailored for fixing LLVM middle-end bugs. Unlike general autofixing agents such as mini-SWE-agent (SWE-agent, 2024; Yang et al., 2024b), our agent is designed based on our real-world experience in fixing LLVM issues leveraging LLVM-specific tools from llvm-autofix.

Basic findings from benchmarking. Our evaluation of frontier models including GPT 5, Gemini 2.5 Pro, DeepSeek V3.2, and Qwen 3 Max using llvm-bench live reveals a reality: *Despite their success on general software bench-*

marks such as SWE-bench Verified (or SWEV), these models struggle with compiler bugs:

- Using mini-SWE-agent integrated with llvm-autofix tools, we observe a drop of 62% in resolution rates on average when models transition from SWEV to llvm-bench live. The best-performing model, DeepSeek V3.2, can resolve only 38% of issues—43%, 32%, and 17% of easy, medium, and hard, respectively—compared to its 60% score in SWEV Leaderboard (SWE-bench, 2024).
- llvm-autofix-mini outperforms mini-SWE-agent by approximately 22% per model. With our agent, the best-performing model is GPT 5, which resolves 52% of issues—59%, 35%, and 21% of easy, medium, and hard, respectively. This highlights the usefulness of llvm-autofix tools and the necessity of developing compiler-specific agents. Yet, its performance still lags behind that in SWEV: 65%.

More findings from expert review. To better understand the performance of frontier models and gain insights into improving llvm-autofix, we recruit an LLVM expert—an active LLVM developer and maintainer—to review the agents’ patches that passed llvm-bench. We observe that *the true capability of frontier models consistently remains below 22%*. In the hard split, only GPT 5 (executed via llvm-autofix-mini) manages to resolve a single issue.

These findings also uncover several technical challenges in order to adopt LLMs for compiler engineering: (1) systematically validating model-generated patches for compilers because our research suggests that LLVM’s regression tests are insufficient, and (2) developing compiler-specific agents utilizing compiler-specific tools or enhancing model expertise in optimizing compilers.

Contributions. Our contributions include:

- A useful agentic harness llvm-autofix that aims to

close the loop between LLMs and compiler engineering, along with a challenging benchmark `llvm-bench`.

- A solid baseline agent `llvm-autofix-mini` for fixing LLVM’s middle-end bugs automatically.
- Insightful findings and open challenges gained from evaluating frontier models and expert review.

2. The `llvm-autofix` Harness

`llvm-autofix` is currently an agentic harness designed to assist auto-fixing agents in understanding and repairing LLVM middle-end bugs, with the goal of covering all types of LLVM bugs. The task is to generate a patch that resolves the bug and successfully passes all relevant tests—LLVM IR programs—utilizing one or more reproducers—also LLVM IR programs—that faithfully replicate the bug.

2.1. Harness Tooling

We wrap common LLVM tasks into agent-accessible tools so that agents can focus on bug repair while the harness handles environment control and validation. This includes tools from the following categories.

Setup & Build. The harness handles build configurations and provides tools for building LLVM with a specific commit and/or target, for example, building for x86 with debug information and assertion checks. The agent can therefore escape from the task of building large systems, a common challenge for autonomous agents (Hu et al., 2025).

Reproduce & Cause. `llvm-autofix` can validate whether the given bug can be triggered faithfully and provide direct causes for reproducible bugs. It sets up LLVM with the bug-containing version, builds it, and runs the reproducer with `opt`—the LLVM middle-end compiler. For crash bugs, such as Figure 1b, it checks whether the crash occurs and provides the stack trace to agents, with non-interesting frames eliminated. For miscompilation bugs, such as Figure 1c, it ensures `opt` exits normally and validates LLVM’s optimization by applying `alive2` (Lopes et al., 2015; 2021)—LLVM’s middle-end translation validator; this will output a counterexample demonstrating the bug if successful.

Explore & Debug. The harness offers searching tools for exploring LLVM’s static and dynamic internals. Static information includes commit-specific code, documentation, and LLVM IR’s specification (LLVM, 2025b) through `grep` and `find`-like tools. Dynamic information enables pausing LLVM and inspecting its intermediate states, for example, reading variables and evaluating expressions at a breakpoint, with respect to a reproducer through the `gdb` debugger.

Edit & Patch. With `llvm-autofix`, agents can edit LLVM, preview and revert edits, and submit edits to the harness.

Test & Validate. The correctness of the submitted patch can be validated and feedback can be provided to agents. `llvm-autofix` will rebuild LLVM and execute relevant tests, typically including the reproducers themselves and regression tests from the associated middle-end component such as `InstCombine`. When permitted, more extensive testing is also available, including regression tests from other components such as `SLPVectorizer` and differential testing of the IR post-optimization if the golden patch is accessible. This validation is supported by various compiler-specific engineering tools, such as `opt`, `alive2`, and `llvm-lit` (or `FileCheck`). We do not include random testing tools such as `csmith` (Yang et al., 2011) because, based on our experience, it is unable to detect bugs within a few hours. Depending on the user preferences, submitted patches can be validated either online (i.e., during the agent execution) or offline (i.e., after the agent execution).

Benchmark & Evaluate. Based on the above tooling, `llvm-autofix` provides an environment and scripts to automatically benchmark and evaluate an agent with our benchmark `llvm-bench`. These scripts output the validated patches (or errors for failed runs), cost, and trajectories.

2.2. The `llvm-bench` Benchmark

We build a benchmark to understand the limitations and improve the capabilities of models or auto-fixing agents.

Construction. We construct `llvm-bench` *automatically* by leveraging the `llvm-autofix` tooling. The construction process consists of three stages: (1) issue collection, (2) reproducer validation, and (3) golden patch validation.

Stage I: Issue collection. We review LLVM’s fixed issues on GitHub and their corresponding commits. For each issue, we filter them based on criteria related to the following fields: *type*, *fixing commit*, *reproducers*, *golden patch*, *base commit*, and some *metadata*. The details of these fields and the filtering process are expanded in Section A.1.

Stage II: Reproducer validation. We validate whether the extracted reproducers can be reproduced on the base commit leveraging `llvm-autofix`’s tools for **Reproduce & Cause**. Collected issues that cannot be reproduced are excluded.

Stage III: Golden patch validation. We validate whether the golden patch is valid by executing `llvm-autofix` tools for **Test & Validate** on the base commit. Collected issues with the golden patch that cannot be validated are excluded.

Statistics. As of Aug. 26, 2025—when we started benchmark construction—we collected 334 issues (example: Section D.1) starting from Jan. 1, 2024.¹ Table 1’s upper half (Name: full) displays the statistics. These issues directly

¹We did not consider issues prior to this date as old issues

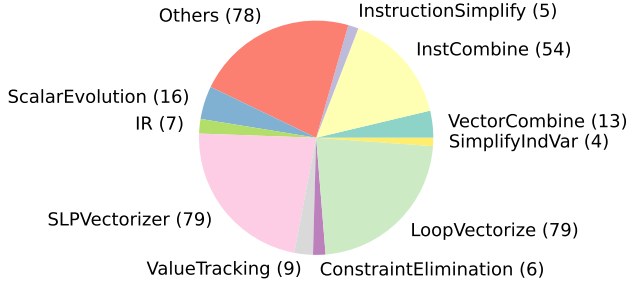


Figure 2. Distribution of Affected Components in 11vm-bench

Table 1. 11vm-bench includes 334 reproducible issues with robust regression tests. “#C/#M” is the number of crash/miscompilation bugs; “#Comp” the number of affected components. “#Rep” and “#Reg” are the average reproducers and component-specific regression tests per issue. “Fixes” report the average edited lines, functions, and files per golden patch.

Name	Split	#Bugs (#C/#M)	#Comp	Tests		Fixes		
				#Rep	#Reg	#Lines	#Funcs	#Files
full	full	334 (222/112)	64	1.4	722.2	17.1	1.6	1.2
	easy	255 (176/79)	48	1.4	734.1	9.1	1.0	1.0
	medium	44 (26/18)	18	1.4	626.5	38.8	2.6	1.0
	hard	35 (20/15)	25	1.7	756.6	47.6	4.9	2.7
live	full	229 (160/69)	43	1.5	743.7	18.5	1.7	1.2
	easy	172 (124/48)	31	1.5	769.8	9.0	1.0	1.0
	medium	34 (21/13)	13	1.4	646.7	41.6	2.6	1.0
	hard	23 (15/8)	20	1.8	691.7	55.7	5.1	2.8

affect 64 out of 184² middle-end components in LLVM, with 176 affected indirectly. Figure 2 presents a breakdown; the full list is displayed in Section A.3. In addition to reproducers and component-specific regression tests, each issue is accompanied by more than 10,000 shared regression tests from other middle-end components. The average size of each reproducer is 49.7 lines of code.

Three splits. Based on the difficulty in resolving these issues, we divide 11vm-bench into three splits: easy issues, which can be resolved by updating a single function; medium issues, which require changes to multiple functions within the same file; and hard issues, which necessitate changes across multiple files in LLVM.

One subset: 11vm-bench live. Evaluating agents across the entire 11vm-bench can be both time-consuming and costly—our experiments indicate that even the best-performing model requires at least 15 minutes and 1.3 million tokens per issue (see Section C.3). Older issues may also become outdated or less useful for the most recent LLMs. Therefore, we create a *continually* updated live subset that always includes 11vm-bench issues from the *lat-*

might be included in the recent LLMs’ training data. Moreover, 11vm-autofix is continually updated: When we submitted this paper, 11vm-bench has been extended to 446 reproducible issues.

²Counted based on LLVM 21.1.0’s -O3 optimization pipeline.

est year. At present (Table 1’s lower half), it has 229 issues starting from Aug. 26, 2024, spanning 43 components with 175 indirectly affected. Given the evolving nature of compilers, such as LLVM, an agent’s performance on 11vm-bench live can reflect its compatibility and effectiveness with the latest compiler versions. Therefore, this subset aims to be a workaround for the data leakage problem—a common challenge for benchmarking LLMs (Deng et al., 2024).

2.3. The 11vm-autofix-mini Agent

To demonstrate the practicality and benefits of our harness, we design a minimal agent that takes advantage of the tooling. Unlike typical auto-fixing agents (Yang et al., 2024b; Mathai et al., 2025) that resolve software bugs based solely on static information from bug reports and software repositories, 11vm-autofix-mini additionally utilizes LLVM’s runtime information obtained through the reproducer. We structure it into a four-stage agent: “Setup → Reason → Generate → Validate”. The first two stages perform root cause analysis to gather information beneficial for the third stage; the final stage handles post-generate, offline patch validation. If the agent unexpectedly exits during the first three stages (e.g., token limit) or if the patch submitted to the final stage is invalid, it results in a failure. Otherwise, a validated patch, which we call *an accepted patch*, is output.

Stage I: Setup. 11vm-autofix-mini configures the environment using 11vm-autofix by validating the reproducibility of the reproducer, launching LLVM under gdb with the reproducer, and pausing it at specific breakpoints. For crash bugs, it pauses before the crashing function; for miscompilation bugs, before the first transformation. These breakpoints are chosen because, based on our experience in fixing LLVM bugs, minimal reproducers usually indicate that the root cause is near these points. This pause allows 11vm-autofix-mini to examine LLVM’s internal state and detect errors by dynamically debugging. This stage also infers the erroneous component, which is then passed to the LLM in the next *Reason* stage prompt and used for component-specific online testing.

Stage II: Reason. Our agent enters a ReAct (Yao et al., 2023) loop to deduce the root cause. Within this loop, it may invoke tools to jump to other frames such as `debug(cmd=frame 3)`, inspect local states such as `eval(expr=WidePhi)`, or review code or docs such as `code(func=llvm::VPTransformState::get)` and `docs(func=llvm::VPTransformState::get)`. The loop concludes once the agent successfully identifies the root cause.

Stage III: Generate. Based on the root cause determined in the previous stage, it initiates a new ReAct loop for patch synthesis. It may use editing tools to amend or reset LLVM such as `edit(file=.../SLPVectorizer,text=...`,

replace=...) and `reset()`, or testing tools for online testing such as `test()`, with the latter offering feedback on failures. The loop ends when online testing is successful.

Stage IV: Validate. This stage consists of offline testing to assess the correctness of the patch generated in the last stage. It outputs the patch for successful validation.

3. Experiment Setup

We evaluate the performance of frontier LLMs on `llvm-bench live` using `llvm-autofix` tools. We do not use `llvm-bench` to maintain a fair, up-to-date evaluation.

Models. We select four frontier models released in late 2025: GPT 5, Gemini 2.5 Pro, DeepSeek V3.2, and Qwen 3 Max. We further select GPT 4o as a baseline model, since it was released about a year prior to the selected frontier models and before the oldest issue in `llvm-bench live`. All models use the latest versions as of Oct. 1, 2025.³

Agents. SWE-bench Verified (OpenAI, 2024; Jimenez et al., 2024) (or SWEV) and mini-SWE-agent (SWE-agent, 2024; Yang et al., 2024b) are the *de facto* benchmark and agent for measuring a model’s agentic capability on software engineering tasks; as far as we know, all frontier models report their results with them. We therefore adopt mini-SWE-agent to `llvm-autofix` as the canonical baseline agent and compare it against `llvm-autofix-mini`.

mini-SWE-agent is given access to all bash tools available in ubuntu:24.04 (e.g., `grep`, `sed`) and `llvm-autofix` (e.g., `opt`, `llvm-lit`, `test`). In addition to the reproducer(s), we further provide mini-SWE-agent with: (1) the inferred erroneous component, (2) the direct cause of the bug, including the stack trace for crash bugs and a counterexample for miscompilation bugs, and (3) the same post-generate validation measures as `llvm-autofix-mini` (Section 2.3). `llvm-autofix-mini` only has access to a subset of `llvm-autofix` tools: `find`, `grep`, `list`, `read`, `edit`, `code`, `docs`, `langref`, `debug`, `eval`, `reset`, `preview`, and `test`.

We do not use larger, production-oriented agents such as Gemini CLI (Gemini, 2025) and Codex (OpenAI, 2025). These agents orchestrate many tools and heuristics, which can *mask* the underlying model behavior: they may fix bugs primarily through extensive scaffolding rather than the model’s own problem-solving ability. However, both mini-SWE-agent and `llvm-autofix-mini` are designed to be minimal, making them suitable for isolating and comparing the models’ inherent agentic skills. Following LLVM’s

policy on AI tools (LLVM, 2025a), we plan to integrate these agents into `llvm-autofix` in the future for real-world LLVM repair and maintenance. The system prompts for both agents are provided in Section E.

Parameters. We use the following settings: temperature 0, context window 64,000 tokens, default reasoning effort, chat limit 500 rounds, token limit 5 million tokens, edit limit 25 calls, and online test limit 25 calls.

Result analyses. We assess whether the agent can produce patches that successfully resolve bugs. As mentioned earlier, a patch is considered an *accepted patch* if it passes all applied online and offline tests, and therefore each resolved issue is associated with one accepted patch. The results are presented as the rate of successfully resolved issues (“% Resolved = # Accepted Patches / # Issues”) according to the `pass@1` metric. We then analyze three aspects:

Benchmark and model performance (Section 4.1). Using mini-SWE-agent, we compare each model’s performance on `llvm-bench live` with its performance on SWEV. This reveals how challenging LLVM middle-end bugs are relative to common software bugs.

Baseline comparison and common failures (Section 4.2). We compare the performance for each model under mini-SWE-agent and `llvm-autofix-mini` to evaluate whether `llvm-autofix-mini` is a stronger baseline than mini-SWE-agent in the compiler engineering setting. For unresolved issues, we categorize their common failures.

Genuine capability via expert review (Section 4.3). Although passing all tests, accepted patches may still be incorrect due to the limitation of dynamic testing. We employ an expert—an active LLVM developer and maintainer—to review every accepted patch from mini-SWE-agent and `llvm-autofix-mini`. The expert labels *correct patches* (semantically sound and review-worthy for LLVM) and categorizes common LLM mistakes among incorrect ones. This yields each model’s *genuine* capability beyond what the previous, automated “% Resolved” alone can capture.

4. Experiment Results

4.1. Benchmark and Model Performance

We first evaluate the capabilities of the widely used mini-SWE-agent in resolving `llvm-bench live` issues and compare its performance with SWEV. As shown in Table 2, fixing LLVM issues proves to be more challenging. Despite having a larger budget, the model’s performance still drops by 62% on average (ranging from 35.2% to 82.9%) when replacing SWEV with `llvm-bench live`. Current LLM’s repair capabilities against LLVM issues, although all improved over GPT 4o, are also not satisfactory. The best

³An exception is DeepSeek V3.2, released on Dec. 1, 2025. Our initial experiments (late Nov. to early Dec.) used DeepSeek V3.2 Exp, the then-current version. A silent update to V3.2 on Dec. 1 caused mixed results, with some data from V3.2 Exp and some from V3.2, so we repeated the experiment using V3.2 only.

Table 2. `llvm-bench live` is a challenging benchmark for current LLMs: Fixing LLVM issues is more difficult than fixing common software issues with `mini-SWE-agent`. “SWEV” denotes SWE-bench Verified; the data are copied verbatim from the SWE-bench (Bash Only) leaderboard or the respective paper. “full”, “easy”, “medium”, and “hard” are different benchmark splits.

Model	% Resolved					\$ Avg. Cost	
	SWEV	full	easy	medium	hard	SWEV	full
GPT 4o	21.6	8.3 (-61.6%)	9.9	5.9	0.0	1.53	1.59
GPT 5	65.0	21.0 (-67.8%)	23.3	17.6	8.7	0.28	0.74
Gemini 2.5 Pro	53.6	9.2 (-82.9%)	10.5	5.9	4.3	0.29	0.81
Qwen 3 Max	69.6	24.4 (-64.9%)	29.1	17.6	0.0	—	4.32
DeepSeek V3.2	60.0	38.9 (-35.2%)	43.0	32.4	17.4	0.03	0.08

Table 3. `llvm-autofix-mini` is a stronger baseline in repairing LLVM issues than `mini-SWE-agent`. Detailed data regarding the cost and overhead of both agents is presented in Section C.3.

Model	mini-SWE-agent		llvm-autofix-mini	
	% Resolved	\$ Avg. Cost	% Resolved	\$ Avg. Cost
GPT 4o	8.3	1.59	12.2 (1.5 $\times$)	2.18
GPT 5	21.0	0.74	51.5 (2.5 $\times$)	0.59
Gemini 2.5 Pro	9.2	0.81	14.4 (1.6 $\times$)	1.16
Qwen 3 Max	24.4	4.32	35.8 (1.5 $\times$)	5.67
DeepSeek V3.2	38.9	0.08	10.5 (0.3 $\times$)	0.15

performing model is DeepSeek V3.2, yet its performance (89/229) is less than 39%; for Gemini 2.5 Pro, only 21 out of 229 (less than 10%) issues are resolved successfully. GPT 5 and Qwen 3 Max fall in between (less than 25%). These data lag behind their performance on SWEV.

By difficulty. The situation becomes worse as the benchmark difficulty increases across splits. Generally, the resolution rate for the medium and hard splits is consistently below the average for the full set, but that for the easy split is higher. The baseline GPT 4o model can handle fewer than 9% of the issues, while for the issues in the hard split, it is unable to handle any of them. The four frontier models can partially handle easy LLVM bugs, underperform in medium ($\sim 30\%$ worse), while struggling or even failing in hard (further $\sim 60\%$ worse). It is interesting to note that Qwen 3 Max’s performance declines sharply when changing to hard, whereas the other models remain smoother.

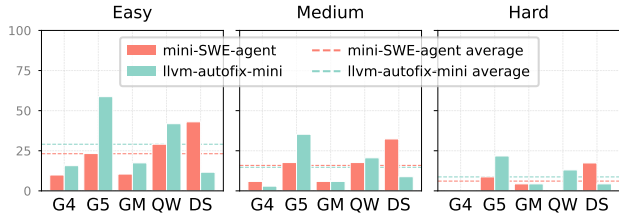


Figure 3. As splits increase in difficulty, frontier models tend to struggle or fail. G4, G5, GM, QW, and DS are short for GPT 4o, GPT 5, Gemini 2.5 Pro, Qwen 3 Max, and DeepSeek V3.2, respectively.

4.2. Baseline Comparison and Common Failures

To investigate the practicality of `llvm-autofix` and the usefulness of `llvm-autofix-mini`, we compare `llvm-autofix-mini` against `mini-SWE-agent`. Table 3 displays the results. In summary, `llvm-autofix-mini`’s performance is generally 1.22 $\times$ of `mini-SWE-agent`’s with affordable cost increment (less than \$1.5 per issue). A statistical test confirming `llvm-autofix-mini`’s significant improvement is presented in Section C.1. The greatest improvement occurs with GPT 5 (+145%), whereas DeepSeek V3.2 is an exception (-73%): when executed with `llvm-autofix-mini`, it frequently fails to adhere to our tool-calling format, whereas such failures are seldom seen with `mini-SWE-agent`; this results in a large (>85%) portion of failures due to exceeding the token budget. The improvement for GPT 4o, Gemini 2.5 Pro, and Qwen 3 Max is 47%, 57%, and 47%, respectively. For all agent-model pairs, GPT 5 successfully resolves the largest number of issues (118/229, 51.5%) when executed with `llvm-autofix-mini`, followed by DeepSeek V3.2 with `mini-SWE-agent` (89/229, 38.9%) and Qwen 3 Max with `llvm-autofix-mini` (82/229, 35.8%). The bug repair capability of the four frontier models is generally better—improved by around 16% when executed with `llvm-autofix-mini`, compared with GPT 4o. The improvement of the GPT, Qwen, and DeepSeek models is more marked than that of the Gemini model.

By difficulty. Although improved over `mini-SWE-agent`, `llvm-autofix-mini`’s performance still lags behind the respective model’s performance in SWEV. The hard split raises a big challenge (Figure 3): Every selected model can fix certain bugs in the easy or medium split, but in the hard split, `llvm-autofix-mini` cannot drive GPT 4o to repair any issue. In particular, `mini-SWE-agent`’s average resolution rate is 23.2%, 15.8%, and 6.1% for easy, medium, and hard, respectively, whereas the data for `llvm-autofix-mini` is 28.8%, 14.7%, and 7.8%, respectively. The degradation for the medium split is primarily due to DeepSeek V3.2 as described earlier. We believe that this indicates the necessity of developing LLVM-specific auto-fixing agents with `llvm-autofix`, rather than relying on a general-purpose agent scaffold such as `mini-SWE-agent`; in this context, `llvm-autofix-mini` serves as a solid baseline.

Common LLM failures. We group failures by LLMs in Figure 4. GPT 4o typically fails due to context limitation (CtxtLimit), because the model frequently calls tools such as `read` to understand the issue, whose output is long, exceeding the context window (64K tokens). The four frontier models rarely have such failures possibly due to their stronger understanding capability. Instead, their failure symptoms vary depending on the underlying agent. For `llvm-autofix-mini`, the majority failures are TokenLimit

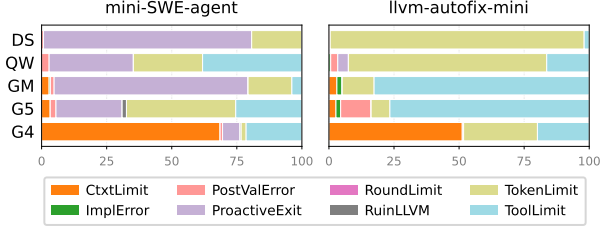


Figure 4. Failure distribution of unresolved issues. G4, G5, GM, QW, and DS are short for GPT 4o, GPT 5, Gemini 2.5 Pro, Qwen 3 Max, and DeepSeek V3.2, respectively.

Table 4. Current LLMs lack satisfactory, genuine capability in fixing LLVM issues: Despite passing all tests, fewer than 42% of accepted patches are correct. “% Correct = # Correct Patches / # Accepted Patches” is the percentage of correct patches with respect to all accepted patches; “% Resolved” indicates the original resolution rate → **genuinely resolved issues after expert review**.

Model	mini-SWE-agent		llvm-autofix-mini	
	% Correct	% Resolved	% Correct	% Resolved
GPT 4o	21.0	8.3 → 1.7	32.1	12.2 → 3.9
GPT 5	31.3	21.0 → 6.6	39.0	51.5 → 20.1
Gemini 2.5 Pro	23.8	9.2 → 2.2	36.4	14.4 → 5.2
Qwen 3 Max	35.7	24.4 → 8.7	36.6	35.8 → 13.1
DeepSeek V3.2	37.1	38.9 → 14.4	41.7	10.5 → 4.4

and ToolLimit, i.e., reaching the limit of token budget (5M tokens) or tool budget (25 **edits** or 25 **tests**). If given a larger budget, we expect llvm-autofix-mini to continue executing. Although TokenLimit and ToolLimit occupy nearly a half as well, a large number of mini-SWE-agent failures are ProactiveExit—it proactively exits the repair process, declaring that it cannot fix the issue. There are cases where mini-SWE-agent ruins the LLVM repository (RuinLLVM), leading to crashes during patch validation; such failures do not happen to llvm-autofix-mini. Post validation failures (PostValError) take up to 12%. Two bugs in llvm-autofix-mini caused several ImplErrors. We did not observe any RoundLimit errors from exceeding the 500-round chat budget.

4.3. Genuine Performance via Expert Review

Table 4 shows the percentage of correct patches with respect to all accepted patches after the LLVM expert’s code review (“% Correct”), as well as the genuine rate of successfully resolved issues (“% Resolved”); in this context, we recognize an issue as being truly resolved if the accepted patch is deemed correct by the expert reviewer.

Despite advancements over GPT 4o, frontier models still do not possess adequate, genuine capabilities for repairing LLVM issues: even for the best-performing llvm-autofix-mini with GPT 5, more than 60% of the accepted patches are incorrect, resulting in only 20.1% issues

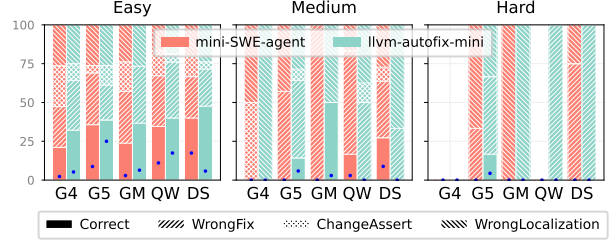


Figure 5. **As splits increase in difficulty, frontier model’s genuine capability degrades fast:** No models can handle the hard split except GPT 5 (llvm-autofix-mini). G4, G5, GM, QW, and DS are short for GPT 4o, GPT 5, Gemini 2.5 Pro, Qwen 3 Max, and DeepSeek V3.2, respectively. Blue bullets (•) represent the rate of genuinely resolved issues, same as “% Resolved” in Table 4.

being genuinely resolved. The results become even worse as the split difficulty increases (Figure 5). Many agent-model pairs (6/10) are unable to output any correct patches for the medium split. In terms of the hard split, only GPT 5 manages to resolve a single issue with llvm-autofix-mini. Despite this, there are cases where the correct patch provided by the model is better than the golden patch provided by the developers; we display such an example in Section D.2.

Common LLM mistakes. We identify three categories of LLM mistakes (with examples presented in Section D.2):

- **ChangeAssert.** LLMs “cheat” the system by altering the assertion condition to avoid assertion failures—one type of crash bugs. This includes (1) changing the assertion itself, (2) modifying functions directly or indirectly invoked by the assertion, and (3) inserting early return statements preceding the assertion.
- **WrongLocalization.** Despite llvm-autofix providing llvm-autofix-mini and mini-SWE-agent with the erroneous component, the models struggle to localize bugs within the erroneous files and functions, particularly when bugs impact multiple functions or files. Detailed data regarding bug localization is in Section C.2.
- **WrongFix.** LLMs generate faulty patches although with correct localization. However, for large, complex systems like LLVM, accepted patches generated in this manner often (1) bypass the erroneous component via weakening or strengthening the activation condition instead of addressing the issue, (2) lack generality beyond reproducers and regression tests, and (3) introduce new bugs silently, such as missed optimizations, where the patch inhibits subsequent optimizations, among others.

4.4. Discussion

We believe our evaluation highlights the value of `llvm-autofix` to the community: Firstly, it underscores the complexity of `llvm-bench`, which we believe is beneficial to understanding LLM’s limitations when applying them to complex software systems (Section 4.1). Secondly, it emphasizes the need for developing LLVM-specific agents, with `llvm-autofix-mini` providing a more solid baseline than `mini-SWE-agent` (Section 4.2). Lastly, our evaluation of the base model and four frontier models shows considerable improvements in the latter, yet they still lack satisfactory proficiency in repairing LLVM bugs compared with repairing common software bugs (Sections 4.1 and 4.3).

Limitation. Section 4.3 implies that `llvm-autofix` is currently limited by LLVM’s regression tests—these tests are inadequate to systematically validate agent-generated patches, especially in detecting the three categories of LLM mistakes. However, unlike common software, addressing such issues poses challenges for compilers; we discuss this later in Section 5. `llvm-autofix` does not currently consider performance issues related to the middle-end, such as slow compilations, compiler hangs, and missed optimizations, nor does it include frontend and backend bugs—however, including them would require the same methodology (Section 2.2) and similar tools such as `arm-tv` (Berger et al., 2025). In fact, we have already incorporated compiler hangs by the time of submitting this paper. We discuss threats that may affect the validity of our results in Section C.4.

5. LLMs for Compilers: Open Challenges

In addition to developing compiler-specific agents using compiler tooling and improving model expertise in compilers (Section 4.1), this work exposes several other open challenges when applying LLMs to compiler engineering, especially regarding online and offline patch validation.

Preventing bypassing. LLMs can manipulate logic by altering conditions or inserting shortcuts that bypass essential checks or optimizations (Section 4.3). In common software, bypassing components leads to errors, whereas in compilers, it results in suboptimal code. Developing lightweight yet robust bypass defenses to ensure assertions are active, components are functioning, and key invariants are upheld presents the first challenge.

Overcoming short-sightedness. LLMs tend to halt repairs when they seem difficult or when current tests pass (Sections 4.2 and 4.3). This often results in incomplete patches that fail to compile or optimize unseen code. Addressing the need for more effective online strategies and systematic offline approaches to pre-detect potential bugs is crucial for

achieving sustained, comprehensive repair.

Managing long context. Compilers’ complexity necessitates large context limits (Section 4.2), but long context can lead to the “context rot” problem (Hong et al., 2025). The final challenge lies in designing sophisticated context management mechanisms that maintain concise yet tractable context, rather than dealing with overloaded contexts.

6. Related Work

Compiler testing. Recent research has increasingly leveraged LLMs to enhance compiler validation and verification. Some works integrate LLMs into the fuzzing process to generate random input programs (Xia et al., 2024; Yang et al., 2024a; Ni & Li, 2025; Munley et al., 2024) and mutation operators (Ou et al., 2025), targeting crash bugs and miscompilation bugs. Other works focus on detecting performance issues such as missed-optimizations (Italiano & Cummins, 2025; Xu et al., 2025). These works demonstrate significant potential, and `llvm-autofix` offers an opportunity to address the bugs they uncover.

Compiler optimization. Integrating traditional ML techniques and LLMs into the compilation pipeline also shows promise for improving both code size and performance. These works usually target specific compiler components, such as inlining (Trofin et al., 2021; Cummins et al., 2025), register allocation (VenkataKeerthy et al., 2023; Cummins et al., 2025), loop optimization (Taneja et al., 2025), pass scheduling (Kulkarni & Cavazos, 2012), auto-tuning (Ashouri et al., 2018), and super optimization (Wei et al., 2025b;a). In this context, CompilerGym provides a toolkit for applying reinforcement learning to compiler optimization tasks (Cummins et al., 2022).

Other systems tasks. Beyond compilers, LLMs have been applied to other systems tasks. For instance, certain studies focus on Linux kernel crash diagnosis and repair, offering benchmarks and environments to evaluate LLM-based agents (Mathai et al., 2024; 2025; Shehada et al., 2025). Additionally, research has explored LLMs’ ability to synthesize efficient GPU kernels (Ouyang et al., 2025).

7. Conclusion

Despite LLMs’ success in common software, they face challenges with compiler-specific issues. To close the loop, our agentic harness `llvm-autofix` offers essential LLVM-specific tools and a challenging benchmark, promoting the development of specialized agents like `llvm-autofix-mini`. Our expert reviews also highlight ongoing challenges, including improving model expertise in compiler optimization and devising robust evaluation methods for patches.

References

- Ashouri, A. H., Killian, W., Cavazos, J., Palermo, G., and Silvano, C. A survey on compiler autotuning using machine learning. *ACM Comput. Surv.*, 2018.
- Berger, R., Briles, M., Boushehrinejad Moradi, N., Coughlin, N., Lam, K., Lopes, N. P., Mada, S., Tirpankar, T., and Regehr, J. Translation validation for LLVM’s AArch64 backend. *Proc. ACM Program. Lang.*, (OOPSLA2), 2025.
- Chen, Z., Tang, R., Deng, G., Wu, F., Wu, J., Jiang, Z., Prasanna, V., Cohan, A., and Wang, X. LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL ’25, 2025.
- Cummins, C., Wasti, B., Guo, J., Cui, B., Ansel, J., Gomez, S., Jain, S., Liu, J., Teytaud, O., Steiner, B., Tian, Y., and Leather, H. CompilerGym: Robust, performant compiler optimization environments for AI research. In *Proceedings of the 2022 20th IEEE/ACM International Symposium on Code Generation and Optimization*, CGO ’22, 2022.
- Cummins, C., Seeker, V., Grubisic, D., Roziere, B., Gehring, J., Synnaeve, G., and Leather, H. LLM Compiler: Foundation language models for compiler optimization. In *Proceedings of the 2025 34th ACM SIGPLAN International Conference on Compiler Construction*, CC ’25, 2025.
- Deng, C., Zhao, Y., Tang, X., Gerstein, M., and Cohan, A. Benchmark probing: Investigating data leakage in large language models. In *Proceedings of the NeurIPS 2023 Workshop on Backdoors in Deep Learning - The Good, the Bad, and the Ugly*, 2024.
- Gemini. Gemini CLI, 2025. URL <https://github.com/google-gemini/gemini-cli>. Accessed: Jan 17th 2026.
- Hong, K., Troynikov, A., and Huber, J. Context rot: How increasing input tokens impacts LLM performance. Technical report, Chroma, 2025.
- Hu, L., Chen, G., Shang, X., Cheng, S., Wu, B., Li, G., Zhu, X., Zhang, W., and Yu, N. CompileAgent: Automated real-world repo-level compilation with tool-integrated LLM-based agent system. In *Proceedings of the 2025 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL ’25, 2025.
- Italiano, D. and Cummins, C. Finding missed code size optimizations in compilers using large language models. In *Proceedings of the 2025 34th ACM SIGPLAN International Conference on Compiler Construction*, CC ’25, 2025.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. R. SWE-bench: Can language models resolve real-world github issues? In *Proceedings of the 2024 12th International Conference on Learning Representations*, ICLR ’24, 2024.
- Kulkarni, S. and Cavazos, J. Mitigating the compiler optimization phase-ordering problem using machine learning. In *Proceedings of the 2012 ACM International Conference on Object Oriented Programming Systems Languages and Applications*, OOPSLA ’12, 2012.
- Li, S., Theodoridis, T., and Su, Z. Boosting compiler testing by injecting real-world code. *Proc. ACM Program. Lang.*, (PLDI), 2024.
- Livinskii, V., Babokin, D., and Regehr, J. Random testing for C and C++ compilers with YARPGen. *Proc. ACM Program. Lang.*, (OOPSLA), 2020.
- Livinskii, V., Babokin, D., and Regehr, J. Fuzzing loop optimizations in compilers for C++ and data-parallel languages. *Proc. ACM Program. Lang.*, (PLDI), 2023.
- LLVM. LLVM AI tool use policy, 2025a. URL <https://llvm.org/docs/AIToolPolicy.html>. Accessed: Jan 17th 2026.
- LLVM. LLVM language reference manual, 2025b. URL <https://llvm.org/docs/LangRef.html>. Accessed: Jan 17th 2026.
- LLVM. LLVM’s analysis and transform passes, 2025c. URL <https://llvm.org/docs/Passes.html>. Accessed: Jan 17th 2026.
- Lopes, N. P., Menendez, D., Nagarakatte, S., and Regehr, J. Provably correct peephole optimizations with Alive. In *Proceedings of the 2015 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI ’15, 2015.
- Lopes, N. P., Lee, J., Hur, C.-K., Liu, Z., and Regehr, J. Alive2: Bounded translation validation for LLVM. In *Proceedings of the 2021 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI ’21, 2021.
- Mathai, A., Huang, C., Maniatis, P., Nogikh, A., Ivancic, F., Yang, J., and Ray, B. kGym: A platform and dataset to benchmark large language models on Linux kernel crash resolution. In *Proceedings of the 2024 38th Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, NeurIPS ’24, 2024.
- Mathai, A., Huang, C., Ma, S., Kim, J., Mitchell, H., Nogikh, A., Maniatis, P., Ivančić, F., Yang, J., and Ray, B. CrashFixer: A crash resolution agent for the Linux kernel, 2025. URL <https://arxiv.org/abs/2504.20412>.

- Munley, C., Jarmusch, A., and Chandrasekaran, S. LLM4VV: Developing LLM-driven testsuite for compiler validation. *Future Gener. Comput. Syst.*, 2024.
- Ni, Y. and Li, S. Interleaving large language models for compiler testing. *Proc. ACM Program. Lang.*, (OOPSLA2), 2025.
- OpenAI. Introducing SWE-bench Verified, 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>. Accessed: Jan 17th 2026.
- OpenAI. Codex, 2025. URL <https://github.com/openai/codex>. Accessed: Jan 17th 2026.
- Ou, X., Li, C., Jiang, Y., and Xu, C. The mutators reloaded: Fuzzing compilers with large language model generated mutation operators. In *Proceedings of the 2024 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, ASPLOS '24, 2025.
- Ouyang, A., Guo, S., Arora, S., Zhang, A. L., Hu, W., Re, C., and Mirhoseini, A. KernelBench: Can LLMs write efficient GPU kernels? In *Proceedings of the 2025 42nd International Conference on Machine Learning, ICML '25*, 2025.
- Shehada, K., Wu, Y., Feng, W. D., Iyer, A., Kumfert, G., Ding, Y., and Qian, Z. Rethinking kernel program repair: Benchmarking and enhancing LLMs with RGym. In *Proceedings of the NeurIPS 2025 Workshop on Evaluating the Evolving LLM Lifecycle: Benchmarks, Emergent Abilities, and Scaling*, 2025.
- Sun, C., Le, V., Zhang, Q., and Su, Z. Toward understanding compiler bugs in gcc and llvm. In *Proceedings of the 2016 25th International Symposium on Software Testing and Analysis*, ISSTA '16, 2016.
- SWE-agent. mini-swe-agent, 2024. URL <https://github.com/SWE-agent/mini-swe-agent>. Accessed: Jan 17th 2026.
- SWE-bench. Official leaderboards (bash only), 2024. URL <https://www.swebench.com/bash-only.html>. Accessed: Jan 17th 2026.
- Taneja, J., Laird, A., Yan, C., Musuvathi, M., and Lahiri, S. K. LLM-Vectorizer: LLM-based verified loop vectorizer. In *Proceedings of the 2025 23rd ACM/IEEE International Symposium on Code Generation and Optimization*, CGO '25, 2025.
- Trofin, M., Qian, Y., Brevdo, E., Lin, Z., Choromanski, K., and Li, D. MLGO: a machine learning guided compiler optimizations framework, 2021. URL <https://arxiv.org/abs/2101.04808>.
- VenkataKeerthy, S., Jain, S., Kundu, A., Aggarwal, R., Cohen, A., and Upadrasta, R. RL4ReAl: Reinforcement learning for register allocation. In *Proceedings of the 2023 32nd ACM SIGPLAN International Conference on Compiler Construction*, CC '23, 2023.
- Wei, A., Suresh, T., Tan, H., Xu, Y., Singh, G., Wang, K., and Aiken, A. Improving assembly code performance with large language models via reinforcement learning. In *Proceedings of the NeurIPS 2025 4th Workshop on Deep Learning for Code*, 2025a.
- Wei, A., Suresh, T., Tan, H., Xu, Y., Singh, G., Wang, K., and Aiken, A. SuperCoder: Assembly program super-optimization with large language models, 2025b. URL <https://arxiv.org/abs/2505.11480>.
- Xia, C. S., Paltenghi, M., Tian, J. L., Pradel, M., and Zhang, L. Fuzz4All: Universal fuzzing with large language models. In *Proceedings of the 2024 46th International Conference on Software Engineering*, ICSE '24, 2024.
- Xia, C. S., Deng, Y., Dunn, S., and Zhang, L. Demystifying LLM-based software engineering agents. *Proc. ACM Softw. Eng.*, (FSE), 2025.
- Xu, Z., Xu, H., Tian, Y., Zhou, X., and Sun, C. Leveraging large language models to detect missed peephole optimizations, 2025. URL <https://arxiv.org/abs/2508.16125>.
- Yang, C., Deng, Y., Lu, R., Yao, J., Liu, J., Jabbarvand, R., and Zhang, L. WhiteFox: White-box compiler fuzzing empowered by large language models. *Proc. ACM Program. Lang.*, (OOPSLA2), 2024a.
- Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K. R., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Proceedings of the 2024 38th Annual Conference on Neural Information Processing Systems*, NeurIPS '24, 2024b.
- Yang, X., Chen, Y., Eide, E., and Regehr, J. Finding and understanding bugs in C compilers. In *Proceedings of the 2011 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '11, 2011.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the 2023 11th International Conference on Learning Representations*, ICLR '23, 2023.

A. llvm-bench: More Details

A.1. Benchmark Construction

We construct llvm-bench *automatically* by leveraging the llvm-autofix tooling. The construction process consists of three stages: (1) issue collection, (2) reproducer validation, and (3) golden patch validation.

Stage I: Issue collection. We review LLVM’s fixed issues on GitHub and the corresponding commits. For each issue, we include the following details.

- *Type*: Either a miscompilation or a crash in the middle-end, indicated by the developer-annotated labels. Issues lacking relevant labels or bearing labels such as wontfix, duplicate, and invalid are omitted.
- *Fixing Commit*: The commit that resolves the issue. In LLVM, valid middle-end fixing commits usually alter code in llvm/lib/ or llvm/include/ and either introduce new tests in llvm/test/ or modify existing tests. Issues are excluded if no such commit exists, if such commit is deemed invalid, or if such commit reverts LLVM to previous commits.
- *Reproducers*: One or more LLVM IR files and associated command line options for opt. These are extracted using llvm-extract based on changes relevant to llvm/test/ within the fixing commit.
- *Golden Patch*: Code changes in llvm/lib/ or llvm/include/, as provided in the fixing commit. Stage III will validate the correctness of this patch.
- *Base Commit*: The commit capable of reproducing the issue. This is the parent commit of the fixing commit.⁴ This commit will undergo validation in Stage II.
- *Metadata*: This includes the issue’s title, description, timestamps, additional labels, and developer discussions. Although this information should not be utilized by auto-fixing agents since it may provide hints on bug fixes, it is invaluable for understanding the issue.

Note that when multiple issues are closed by the same commit, only the oldest issue is retained; recent issues typically refer to cases of duplication that lack appropriate duplicate labels due to developers’ oversight.

Stage II: Reproducer validation. We validate whether the extracted reproducers can be reproduced on the base commit leveraging llvm-autofix’s tools for **Reproduce & Cause** (Section 2.1). Issues that cannot be reproduced are excluded.

Stage III: Golden patch validation. We validate whether the golden patch is valid by executing llvm-autofix tools for **Test & Validate** (Section 2.1) on the base commit. Issues with the golden patch that cannot be validated are excluded.

A.2. Benchmark Statistics

As of Aug. 26, 2025—when we started benchmark construction—we successfully collected 334 issues starting from Jan. 1, 2024.⁵ This includes 222 crashes and 112 miscompilations. An example crash issue is presented in Section D.1. Table 1’s upper half (Name: full) displays the statistics.

Affected components. These issues directly affect a total of 64 out of 184⁶ middle-end components in LLVM, with 176 components affected indirectly. LoopVectorize, SLPVectorizer, and InstCombine—three components known to be the most erroneous (Yang et al., 2011; Livinskii et al., 2020; 2023; Li et al., 2024)—are affected by the greatest number of issues. Figure 2 presents a breakdown; the full list is displayed in Section A.3. These components span 937 C++ files and 676,308 lines of LLVM code. Currently, the benchmark lacks issues from other components, as these issues predate Jan. 1, 2024.

⁴If the parent commit is not buildable, we trace the git history of the main branch to identify the most recent buildable commit and update the golden patch and reproducers accordingly.

⁵We did not consider issues prior to this date as old issues might be included in the recent LLMs’ training data. Moreover, llvm-autofix is continually updated: When we submitted this paper, llvm-bench has been extended to 446 reproducible issues.

⁶Counted based on LLVM 21.1.0’s -O3 optimization pipeline.

Reproducers & tests. Each issue is on average coupled with 1.41 reproducers, 722 component-specific regression tests, and more than 10,000 regression tests from other components. Reproducers and regression tests are minimal LLVM IR programs reduced by automatic tools and LLVM developers. Each reproducer is 49.74 lines of code.

Fixes & patches. Resolving these LLVM issues is challenging. When LLVM developers initially submitted these patches, fewer than 50% were accepted without further bug fixes; the remainder required correction based on reviewer’s feedback and subsequent resubmission. In the end, every golden patch provided by LLVM developers involves the editing of around 17.06 lines, 1.62 functions, and 1.17 files. Among all issues, 255 (76.3%) could be fixed by updating one function, 44 (13.2%) by updating multiple functions within one file, and the rest 35 (10.5%) need multiple files; these data indicate the difficulty of fixing these issues, and thereby the benchmark can be divided into three splits: easy, medium, and hard (see Table 1).

A.3. Affected Middle-End Components

SLPVectorizer (79), LoopVectorize (79), InstCombine (54), ScalarEvolution (16), VectorCombine (13), ValueTracking (9), IR (7), ConstraintElimination (6), InstructionSimplify (5), SimplifyIndVar (4), MemorySSAUpdater (4), LoopPeel (4), LoopAccessAnalysis (3), Local (3), GVN (3), FunctionAttrs (3), ConstantFold (3), SimplifyCFG (2), LoopStrengthReduce (2), LoopSimplifyCFG (2), LazyValueInfo (2), LICM (2), GlobalOpt (2), EarlyCSE (2), DeadStoreElimination (2), VectorUtils (1), ValueMapper (1), SimplifyLibCalls (1), SimpleLoopUnswitch (1), SeparateConstOffsetFromGEP (1), Scalarizer (1), SCCPSolver (1), SCCP (1), Reassociate (1), PredicateInfo (1), NewGVN (1), MoveAutoInit (1), MemCpyOptimizer (1), LowerSwitch (1), LoopUnrollRuntime (1), LoopUnrollAndJamPass (1), LoopFuse (1), LoopDeletion (1), LoopCacheAnalysis (1), Loads (1), JumpThreading (1), Instrumentation (1), InlineCost (1), InductiveRangeCheckElimination (1), IndVarSimplify (1), GVNSink (1), FunctionSpecialization (1), Evaluator (1), DemoteRegToStack (1), DeadArgumentElimination (1), DFAJumpThreading (1), CorrelatedValuePropagation (1), Coroutines (1), BreakCriticalEdges (1), BasicBlockUtils (1), BDCE (1), Attributor (1), AliasAnalysis (1), AggressiveInstCombine (1).

B. llvm-autofix-mini: More Details

B.1. Enabled Tools in Evaluation

Type	Signature	Description
Generic	<code>find(pat: str, dir: str)</code>	Find files matching the pattern pat in the directory dir recursively
	<code>grep(args: str)</code>	Find text matching patterns and files mentioned in arguments args
	<code>list(dir: str)</code>	List files and directories in the directory dir
	<code>read(file: str, pos: int, k: int)</code>	Read k lines of content starting from line pos of the file file
	<code>edit(file: str, text:str, replace: str)</code>	Replace the text text with the text replace in the file file
LLVM Specific	<code>preview()</code>	Preview LLM’s code changes in the format of unified diff
	<code>code(func: str)</code>	List the code of the LLVM function func
	<code>docs(func: str)</code>	Show the documentation of the LLVM function func
	<code>langref(inst: str)</code>	Show the language reference of the LLVM IR instruction inst
	<code>debug(cmd: str)</code>	Run GDB command cmd over the current LLVM internal state
	<code>eval(expr: str)</code>	Evaluate the LLVM internal state by the expression expr
	<code>reset()</code>	Reset the LLVM project to its initial state of this bug
	<code>test()</code>	Test the LLM’s patch with LLVM online and return the feedback

C. More Evaluation Data

C.1. Deeper Analysis

llvm-autofix-mini vs mini-SWE-agent. Section 4.2 demonstrates that llvm-autofix-mini outperforms mini-SWE-agent for the three frontier models: GPT 5, Gemini 2.5 Pro, and Qwen 3 Max. To test this assumption statistically, we conducted a one-sided McNemar’s test with a significance level of $\alpha = 0.05$. The results, presented in Table 5, confirm our assumption. It is noteworthy that the p -value for each model decreases as the model’s resolution rate on llvm-autofix-mini increases, where GPT 5’s p -value is close to zero—less than 0.00005.

By bug types. Figure 6 illustrates the "% Resolved" for both agents across different bug types, prior to the expert review.

Table 5. **With frontier models, llvm-autofix-mini significantly outperforms mini-SWE-agent.** Matrix #ij: i and j denote whether mini-SWE-agent and llvm-autofix-mini pass or fail for a particular issue, respectively. Significance level: $\alpha = 0.05$.

Model	#01	#10	#11	#00	χ^2	p-value	Significant?
GPT 5	77	7	41	104	56.68	0.0000	✓
Gemini 2.5 Pro	23	11	10	185	3.56	0.0296	✓
Qwen 3 Max	42	16	40	131	10.78	0.0005	✓

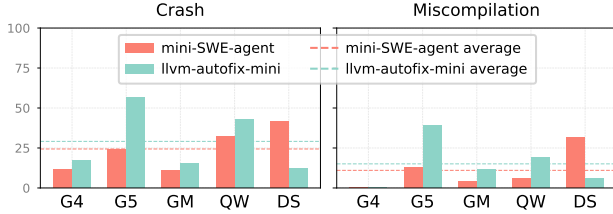


Figure 6. **Miscompilations are more challenging than crashes.** The Y-axis is “% Resolved” before expert review. G4, G5, GM, QW, and DS are short for GPT 4o, GPT 5, Gemini 2.5 Pro, Qwen 3 Max, and DeepSeek V3.2, respectively.

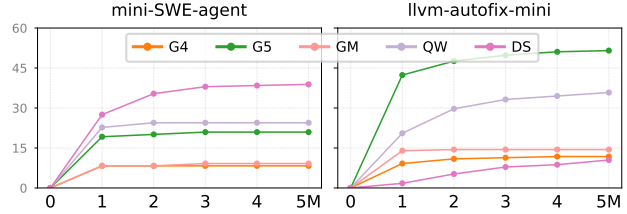


Figure 7. **llvm-autofix-mini is more robust as token limit increases than mini-SWE-agent.** The Y-axis is “% Resolved” before expert review. When executed with mini-SWE-agent, nearly all models converge after reaching three million tokens.

Generally, miscompilations are more challenging to diagnose and repair than crashes, which aligns with our experience. The resolution rate of miscompilations lags approximately 13% and 14% behind crashes for llvm-autofix-mini and mini-SWE-agent, respectively. The baseline model GPT 4o is unable to resolve any miscompilations, whether using mini-SWE-agent or llvm-autofix-mini. Among frontier models, GPT 5 is the best-performing model, whereas Gemini 2.5 Pro’s capability consistently falls below average for either bug type. Regarding agents, llvm-autofix-mini consistently outperforms mini-SWE-agent in addressing both crashes and miscompilations, except with DeepSeek V3.2 due to its failure to adhere to our tool calling format.

With token limit. We investigate whether the agent’s performance improves as token limits increase. Figure 7 displays this trend. mini-SWE-agent’s performance appears to converge when the token limit reaches three million. Although llvm-autofix-mini shows a similar pattern with the baseline model GPT 4o and one frontier model, Gemini 2.5 Pro, llvm-autofix-mini is more robust: the performance of the other three frontier models increases smoothly. It is interesting to note that the performance of DeepSeek V3.2 increases more markedly than other models when executed with llvm-autofix-mini, and it is expected to surpass GPT 4o as the limit extends to six million tokens. An explanation for this could be that DeepSeek V3.2 progressively adheres to our tool calling format.

However, the performance of these models does not notably improve with an increased number of tokens. We propose two hypotheses: 1. The “context rot” problem (Hong et al., 2025) becomes more pronounced as agents operate for extended periods (resulting in increased context). 2. The complexity of compiler bugs makes it increasingly infeasible to resolve the remaining issues within a reasonable token limit.

Tool call distribution. Figure 8 displays the average number of tool invocations per issue. Notably, LLVM-specific tools, particularly the debugger (`eval` and `debug`) are called more frequently than generic tools, highlighting their utility in assisting agents to comprehend the bugs. Within this category, `langref` is rarely invoked, potentially due to the models’ familiarity with LLVM IR. Regarding the `docs` tool, we identified a bug in our implementation of this tool, which prevents models from utilizing it; `code` can show the inline documentation as well. For generic tools, `list` and `find` are rarely used. Although we have wrapped the underlying tools with an agent-friendly interface, frontier models encounter difficulties in invoking `edit`, `eval`, and `debug`, evidenced by a high failure rate, indicating the need for future optimizations.

C.2. Bug Localization

We analyze whether the agent can accurately localize the bug to the erroneous files or functions, i.e., those edited in the golden patch. We parse the most recent patch input to the `test` tool, under the assumption that subsequent generated patches are more refined than earlier ones. Following existing work (Xia et al., 2025; Chen et al., 2025), we report the *recall* at the file level and the function level.

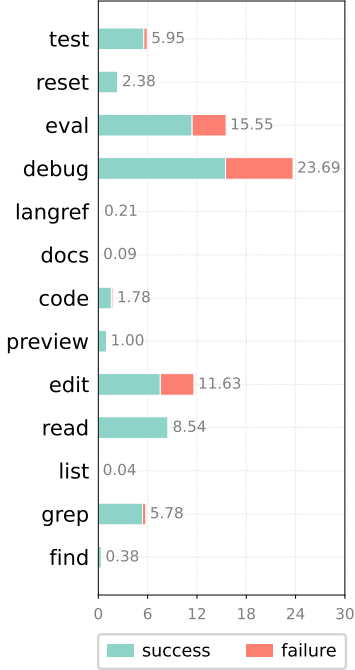


Figure 8. Tool Call Distrib. (Per Issue)

Table 6. Bug Localization Results in Recall

Split	Model	mini-SWE-agent		llvm-autofix-mini	
		% File	% Func	% File	% Func
full	GPT 4o	59.2	29.9	42.5	21.8
	GPT 5	61.4	32.7	76.3	42.4
	Gemini 2.5 Pro	55.0	23.8	48.6	23.6
	Qwen 3 Max	64.2	36.0	64.4	35.6
	DeepSeek V3.2	64.5	36.2	19.6	13.7
easy	GPT 4o	64.0	34.9	48.3	27.3
	GPT 5	65.1	36.6	81.4	48.8
	Gemini 2.5 Pro	59.3	26.7	52.3	26.7
	Qwen 3 Max	69.8	40.1	66.3	41.3
	DeepSeek V3.2	69.2	41.9	20.9	16.3
medium	GPT 4o	61.8	20.1	29.4	6.9
	GPT 5	64.7	22.8	82.4	26.2
	Gemini 2.5 Pro	55.9	17.6	52.9	18.4
	Qwen 3 Max	64.7	30.1	85.3	26.5
	DeepSeek V3.2	64.7	23.3	20.6	5.9
hard	GPT 4o	20.1	6.8	18.6	2.2
	GPT 5	28.3	17.8	28.9	17.9
	Gemini 2.5 Pro	21.2	11.0	14.6	7.7
	Qwen 3 Max	22.0	13.4	19.7	6.6
	DeepSeek V3.2	29.3	12.9	8.0	6.2

Table 6 presents the results. In a nutshell, the current state of LLMs is still far from satisfactory performance for localizing LLVM middle-end bugs, even though the erroneous component has been provided in the prompt. In particular, although released one year later, the bug localization performance of the four frontier models does not advance significantly compared with the baseline model GPT 4o; the improvement at the file and function level is below 10% and 8% on average, respectively. It is also interesting that the performance of Gemini 2.5 Pro with mini-SWE-agent and DeepSeek V3.2 with llvm-autofix-mini are even worse than GPT 4o with the respective agent. Among all selected models, only GPT 5 is able to recall more than 65% of the erroneous files and more than 37% of the erroneous functions when executed with llvm-autofix-mini. However, the respective data are still below 77% and 43%. For Gemini 2.5 Pro and Qwen 3 Max, executing them with llvm-autofix-mini or with mini-SWE-agent leads to comparable recall at either level. DeepSeek V3.2 is an exception: when executed with mini-SWE-agent, its recall at either level is more than 20% better than with llvm-autofix-mini. The reason is that it usually fails to follow llvm-autofix-mini’s tool calling format while such failures are rarely observed with mini-SWE-agent.

C.3. Cost and Overhead

Fixing LLVM issues necessitates more cost and runtime overhead compared with common software issues such as SWEV; see Table 2 and Table 7.

mini-SWE-agent’s results are obtained under a larger budget—5 million tokens—on llvm-bench live than on SWEV—usually around 500 thousand tokens according to Agentless (Xia et al., 2025). In our parameter settings, the average overhead for fixing an issue with the four frontier models is approximately 14.1 minutes (ranging from 7.1 to 18.1 minutes). Despite this, the performance still degrades.

We also observed that llvm-autofix-mini generally calls `test` fewer times than mini-SWE-agent. This results in a better resolution rate, although it involves more rounds to reason and gather information.

C.4. Threats to Validity

There are several potential threats that need clarification. For SWE-bench Verified, we report numbers from its leaderboard to provide context, rather than rerunning the experiment in our environment to maintain controlled measurement. This

Table 7. Cost and Overhead of llvm-autofix-mini and mini-SWE-agent

Model	Time (m)	# Tests	# Rounds	Token Cost (million)					\$ Cost
				# Total	# Input	# Output	# Cached	% Cached	
mini-SWE-agent									
GPT 4o	7.1	13.0	74.4	2.2	2.2	6.9K	2.2	96.4	1.6
GPT 5	14.6	9.5	66.9	2.6	2.6	28.2K	2.5	94.4	0.7
Gemini 2.5 Pro	8.8	7.4	51.7	1.6	1.6	21.3K	1.4	85.8	0.8
Qwen 3 Max	18.1	13.1	77.7	2.4	2.4	15.3K	1.1	43.9	4.3
DeepSeek V3.2	15.1	3.9	80.0	2.3	2.3	17.2K	2.2	97.9	0.1
llvm-autofix-mini									
GPT 4o	15.5	10.4	116.5	3.1	3.1	7.7K	3.0	96.3	2.2
GPT 5	16.1	6.4	39.0	1.3	1.3	32.3K	1.2	90.0	0.6
Gemini 2.5 Pro	12.5	3.5	53.2	1.5	1.5	21.5K	1.3	83.3	1.2
Qwen 3 Max	25.5	8.0	81.4	3.4	3.4	16.8K	1.7	49.3	5.7
DeepSeek V3.2	21.8	1.3	183.5	4.7	4.7	16.3K	4.6	98.5	0.2

Table 8. The data leakage problem does not appear to have a substantial impact on GPT 5 and Gemini 2.5 Pro when fixing LLVM issues. “# Post (e/m/h)” indicates the number of issues (easy/medium/hard) that were fixed post the release date of the respective model. “Pre” and “Post” denote the resolution rate on issues pre- and post-release, respectively. “% Expert” is “% Resolved” after the expert review.

Model	Release Date	# Post (e/m/h)	mini-SWE-agent				llvm-autofix-mini			
			% Resolved		% Expert		% Resolved		% Expert	
			Pre	Post	Pre	Post	Pre	Post	Pre	Post
GPT 4o	2024-07-18	229 (172/34/23)	—	8.3	—	1.7	—	12.2	—	3.9
GPT 5	2025-08-07	14 (9/3/2)	20.9	21.4	7.0	0.0	49.3	85.7	19.5	28.6
Gemini 2.5 Pro	2025-07-17	21 (14/4/3)	9.1	9.5	2.4	0.0	13.5	23.8	4.8	9.5
Qwen 3 Max	2025-09-05	0 (0/0/0)	24.5	—	8.7	—	35.8	—	13.1	—
DeepSeek V3.2	2025-12-01	0 (0/0/0)	38.9	—	14.4	—	10.5	—	4.4	—

follows established practices in academia (Xia et al., 2025; Chen et al., 2025) and industry (i.e., the official model release blog). Therefore, the cross-benchmark comparison (Section 4.1) should be interpreted qualitatively. The next threat relates to our sampling once to report the pass@1 resolution rate. Due to budget limitations, we are unable to conduct the experiments for more times. We mitigate this threat by using greedy decoding for each model call. Thirdly, we employ only one LLVM expert to review accepted patches. However, this is the standard code review procedure in the LLVM community for handling the middle-end, where a pull request can be merged upstream after being reviewed and approved by one expert. We also classify accepted patches into different groups when presenting to the expert, to reduce the expert’s review burden. Finally, there is a possibility of data leakage. We initially conduct the experiment using the llvm-bench live subset, which includes only the issues from the most recent year. Additionally, we examine the resolution rate by including only issues that were fixed after the release of each model. The results are shown in Table 8. It is interesting to note that GPT 5 and Gemini 2.5 Pro perform better on issues post their release date, whether using mini-SWE-agent or llvm-autofix-mini. However, after expert review, all mini-SWE-agent-generated accepted patches are deemed incorrect. The genuine resolution rate of llvm-autofix-mini on post-release issues is higher. In summary, models’ performance on post-release issues is comparable to, or better than, on pre-release issues. Therefore, the potential data leakage problem does not appear to have a substantial impact, especially on llvm-autofix-mini. This may be due to the inherent difficulty in fixing compiler bugs.

D. Examples

D.1. An Example Issue

```

1 BugType: Crash
2 BaseCommit: 9a8b0407fc16af4ca6f79a2583297318a645d88a
3 FixingCommit: 3cb82f49dc990dc20a765856c0e126193992fe44
4 Reproducers: ````llvm
5 ; opt -S --passes=slp-vectorizer -mtriple=s390x-unknown-linux-gnu -mcpu=z16 -slp-threshold=-10 < %s
6 define i1 @test(i64 %0, i64 %1, ptr %2) {

```

```

7 entry:
8   %gep44 = getelementptr i8, ptr null, i64 %0
9   %gep45 = getelementptr i8, ptr null, i64 %1
10  %4 = icmp ult ptr %gep44, %gep45
11  %umin = select i1 %4, ptr %gep44, ptr %gep45
12  %gep48 = getelementptr i8, ptr null, i64 %0
13  %gep49 = getelementptr i8, ptr null, i64 %1
14  %5 = icmp ult ptr %gep48, %gep49
15  %umin50 = select i1 %5, ptr %gep48, ptr %gep49
16  %b095 = icmp ult ptr %umin, %2
17  %b196 = icmp ult ptr %umin50, %2
18  %res = and i1 %b095, %b196
19  ret i1 %res
20 }
21 ...
22 GoldenPatch: ```diff
23 Author: Alexey Bataev <a.bataev@outlook.com>
24 Date: Mon Jul 22 12:45:28 2024 -0700
25
26 [SLP]Fix PR99899: Use canonical type instead of original vector of ptr.
27
28 Use adjusted canonical integer type instead of the original ptr type to
29 fix the crash in the TTI.
30 Fixes https://github.com/llvm/llvm-project/issues/99899
31
32 diff --git a/llvm/lib/Transforms/Vectorize/SLPVectorizer.cpp
33       b/llvm/lib/Transforms/Vectorize/SLPVectorizer.cpp
34 index 667c4eb311c2..cca9eeebaa53 100644
35 --- a/llvm/lib/Transforms/Vectorize/SLPVectorizer.cpp
36 +++ b/llvm/lib/Transforms/Vectorize/SLPVectorizer.cpp
37 @@ -9699,7 +9699,8 @@ BoUpSLP::getEntryCost(const TreeEntry *E, ArrayRef<Value *> VectorizedVals,
38     CanonicalType = CanonicalType->getWithNewType(IntegerType::get(
39         CanonicalType->getContext(),
40         DL->getTypeSizeInBits(CanonicalType->getScalarType())));
41 -    IntrinsicCostAttributes CostAttrs(MinMaxID, VecTy, {VecTy, VecTy});
42 +    IntrinsicCostAttributes CostAttrs(MinMaxID, CanonicalType,
43 +        {CanonicalType, CanonicalType});
44 +    InstructionCost IntrinsicCost =
45         TTI->getIntrinsicInstrCost(CostAttrs, CostKind);
46     // If the selects are the only uses of the compares, they will be
47     ...
48 IssueID: 99899
49 IssueTimestamp: 2024-07-22T17:02:03Z
50 IssueAuthor: JonPsson1
51 IssueLabels: llvm:SLPVectorizer, crash-on-valid
52 IssueTitle: [SLP] crash after 8ff233f
53 IssueBody: ```plain
54 [SLP]Correctly detect minnum/maxnum patterns for select/cmp operations on floats." seems to have introduced
55     a problem when building SPEC on SystemZ.
56 opt -mtriple=s390x-linux-gnu -mcpu=z16 -O3 ./tc_slp.ll -o /dev/null
57 opt: /home/ijonpan/llvm-project/llvm/include/llvm/IR/DerivedTypes.h:704: llvm::Type*
58     llvm::Type::getWithNewBitWidth(unsigned int) const: Assertion `isIntOrIntVectorTy() && "Original type
59     expected to be a vector of integers or a scalar integer.'" failed.
60 ...
61 #15 0x0000000000519c8f8 llvm::slpvectorizer::BoUpSLP::getEntryCost
62 [tc_slp.ll.tar.gz](https://github.com/user-attachments/files/16337301/tc_slp.ll.tar.gz)
63 ```

```

D.2. Accepted Patch Examples

Correct. Below is a patch generated by `llvm-autofix-mini` (GPT 5) for fixing a miscompilation in the `InstCombine` component. This patch has been reviewed and deemed correct by the LLVM expert. The expert also noted that this correct patch produces better-optimized code than the golden patch concerning the reproducer.

```

1 diff --git a/llvm/lib/Transforms/InstCombine/InstCombineAndOrXor.cpp
2       b/llvm/lib/Transforms/InstCombine/InstCombineAndOrXor.cpp
3 index 64bee4ab974e..4fd7c2238f21 100644
4 --- a/llvm/lib/Transforms/InstCombine/InstCombineAndOrXor.cpp

```



```

5 +++ b/llvm/lib/Transforms/InstCombine/InstCombineAndOrXor.cpp
6 @@ -3358,19 +3358,34 @@ Value *InstCombinerImpl::foldAndOrOfICmps(ICmpInst *LHS, ICmpInst *RHS,
7     foldAndOrOfICmpEqConstantAndICmp(LHS, RHS, IsAnd, IsLogical, Builder))
8     return V;
9     // We can treat logical like bitwise here, because both operands are used on
10 - // the LHS, and as such poison from both will propagate.
11 + // the LHS, and as such poison from both will propagate. However, if this
12 + // causes us to drop short-circuiting and return a single unconditional
13 + // compare, we must clear the 'samesign' flag to avoid introducing poison
14 + // where the original logical form was guarded.
15 + if (IsLogical) {
16 +     if (auto *IC0 = dyn_cast<ICmpInst>(LHS); IC0 && IC0->hasSameSign())
17 +         IC0->setSameSign(false);
18 +     if (auto *IC1 = dyn_cast<ICmpInst>(RHS); IC1 && IC1->hasSameSign())
19 +         IC1->setSameSign(false);
20 + }
21     if (Value *V = foldAndOrOfICmpEqConstantAndICmp(RHS, LHS, IsAnd,
22 - //IsLogical*/ false, Builder))
23 + //IsLogical*/ false, Builder)) {
24     return V;
25 + }
26
27     if (Value *V =
28         foldAndOrOfICmpsWithConstEq(LHS, RHS, IsAnd, IsLogical, Builder, Q))
29         return V;
30     // We can convert this case to bitwise and, because both operands are used
31 - // on the LHS, and as such poison from both will propagate.
32 + // on the LHS, and as such poison from both will propagate. Same caveat as
33 + // above: if we end up with an unconditional compare, drop 'samesign'.
34     if (Value *V = foldAndOrOfICmpsWithConstEq(RHS, LHS, IsAnd,
35 - //IsLogical*/ false, Builder, Q))
36 + //IsLogical*/ false, Builder, Q)) {
37 +     if (IsLogical)
38 +         if (auto *IC = dyn_cast<ICmpInst>(V); IC && IC->hasSameSign())
39 +             IC->setSameSign(false);
40     return V;
41 + }
42
43     if (Value *V = foldIsPowerOf2OrZero(LHS, RHS, IsAnd, Builder, *this))
44         return V;
45     @@ -3514,7 +3529,17 @@ Value *InstCombinerImpl::foldAndOrOfICmps(ICmpInst *LHS, ICmpInst *RHS,
46     }
47 }
48
49 - return foldAndOrOfICmpsUsingRanges(LHS, RHS, IsAnd);
50 + if (Value *Res = foldAndOrOfICmpsUsingRanges(LHS, RHS, IsAnd)) {
51 +     // For logical select-form folds, if we collapse to a single compare that
52 +     // retains a 'samesign' flag, it may now execute unconditionally whereas the
53 +     // original flagged compare's result was only used conditionally. To avoid
54 +     // introducing poison, conservatively drop the flag.
55 +     if (IsLogical)
56 +         if (auto *IC = dyn_cast<ICmpInst>(Res); IC && IC->hasSameSign())
57 +             IC->setSameSign(false);
58 +     return Res;
59 + }
60 + return nullptr;
61 }
62
63 static Value *foldOrOfInversions(BinaryOperator &I,

```

WrongFix. Below is a patch generated by mini-SWE-agent (Qwen 3 Max) for fixing a miscompilation in the ConstraintElimination component. This patch addresses the reproducer's specific patterns, lacking generality for unseen code.

```

1 diff --git a/llvm/lib/Transforms/Scalar/ConstraintElimination.cpp
2     b/llvm/lib/Transforms/Scalar/ConstraintElimination.cpp
3 index d2e532d29d07..339f5995b74e 100644
4 --- a/llvm/lib/Transforms/Scalar/ConstraintElimination.cpp
5 +++ b/llvm/lib/Transforms/Scalar/ConstraintElimination.cpp
6 @@ -1399,6 +1399,31 @@ static bool checkAndReplaceCondition(

```

```

7   Instruction *ContextInst, Module *ReproducerModule,
8   ArrayRef<ReproducerEntry> ReproducerCondStack, DominatorTree &DT,
9   SmallVectorImpl<Instruction *> &ToRemove) {
10 + BasicBlock *BB = Cmp->getParent();
11 + // Specific fix for the test case:
12 + // - Block has multiple predecessors
13 + // - Condition is 'icmp sgt i16 %n, 0'
14 + // - One predecessor is the entry block
15 + if (std::distance(pred_begin(BB), pred_end(BB)) > 1) {
16 +     Function *F = BB->getParent();
17 +     BasicBlock *EntryBB = &F->getEntryBlock();
18 +
19 +     bool HasEntryPred = false;
20 +     for (BasicBlock *Pred : predecessors(BB)) {
21 +         if (Pred == EntryBB) {
22 +             HasEntryPred = true;
23 +             break;
24 +         }
25 +     }
26 +
27 +     if (HasEntryPred &&
28 +         Cmp->getPredicate() == ICmpInst::ICMP_SGT &&
29 +         Cmp->getType()->isIntegerTy(1) &&
30 +         Cmp->getOperand(1) == ConstantInt::get(Cmp->getOperand(0)->getType(), 0)) {
31 +         return false;
32 +     }
33 + }
34 +
35 auto ReplaceCmpWithConstant = [&](CmpInst *Cmp, bool IsTrue) {
36     generateReproducer(Cmp, ReproducerModule, ReproducerCondStack, Info, DT);
37     Constant *ConstantC = ConstantInt::getBool(

```

ChangeAssert. Below is a patch generated by mini-SWE-agent (DeepSeek V3.2) for fixing a crash in the MemorySSAUpdater component. This patch inserts early-exit code to bypass the subsequent assertion check (Line 13) when matching patterns from the reproducers.

```

1 diff --git a/llvm/lib/Analysis/MemorySSAUpdater.cpp b/llvm/lib/Analysis/MemorySSAUpdater.cpp
2 index ecfecb03c375..72383c0c5a93 100644
3 --- a/llvm/lib/Analysis/MemorySSAUpdater.cpp
4 +++ b/llvm/lib/Analysis/MemorySSAUpdater.cpp
5 @@ -501,6 +501,10 @@ void MemorySSAUpdater::fixupDefs(const SmallVectorImpl<WeakVH> &Vars) {
6     "Should have already handled phi nodes!");
7     // We are now this def's defining access, make sure we actually dominate
8     // it
9 +     if (!MSSA->dominates(NewDef, FirstDef)) {
10 +         // NewDef does not dominate FirstDef, treat this block as having no defs
11 +         break;
12 +     }
13     assert(MSSA->dominates(NewDef, FirstDef) &&
14           "Should have dominated the new access");

```

WrongLocalization. Below is a patch generated by llvm-autofix-mini (GPT 5) for fixing a crash in the ValueTracking component. GPT 5 localizes the bug into FunctionAttrs.cpp, whereas the erroneous file is ValueTracking.cpp.

```

1 diff --git a/llvm/lib/Transforms/IPO/FunctionAttrs.cpp b/llvm/lib/Transforms/IPO/FunctionAttrs.cpp
2 index 56bfc8432cbb..eec21526983f 100644
3 --- a/llvm/lib/Transforms/IPO/FunctionAttrs.cpp
4 +++ b/llvm/lib/Transforms/IPO/FunctionAttrs.cpp
5 @@ -122,6 +122,13 @@ static void addLocAccess(MemoryEffects &ME, const
6 MemoryLocation &Loc,
7     return;
8
9     const Value *UO = getUnderlyingObjectAggressive(Loc.Ptr);
10 + // getUnderlyingObjectAggressive may return null (e.g., due to poison/undef
11 + // or cyclic PHIs). Conservatively treat as non-identified/unknown.
12 + if (!UO) {
13 +     ME |= MemoryEffects::argMemOnly(MR);
14 +     ME |= MemoryEffects::IRMemLocation::Other, MR);
15 +     return;

```

```

16 + }
17   if (isa<AllocaInst>(UO))
18       return;
19   if (isa<Argument>(UO)) {

```

E. Prompts

E.1. System Prompt Example: llvm-autofix-mini's Reason Stage

You are a senior LLVM maintainer, responsible for the `instsimplify` module. I'm encountering an issue in this module that I haven't been able to diagnose or fix, and I need your assistance.

Reproducer

```

```llvm
define half @fabs_select_fabs(half noundef %x) {
entry:
 %abs1 = call half @llvm.fabs.f16(half %x)
 %cmp = fcmp ogt half %abs1, 0xH0000
 %sel = select i1 %cmp, half %x, half 0xH0000
 %abs2 = call half @llvm.fabs.f16(half %sel)
 ret half %abs2
}

; Function Attrs: nocallback nofree nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nofree nosync nounwind speculatable willreturn memory(none) }
```

```

Symptom

```

---
--- src ---
define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = call half @llvm.fabs.f16(half %x)
    %cmp = fcmp ogt half %abs1, 0xH0000
    %sel = select i1 %cmp, half %x, half 0xH0000
    %abs2 = call half @llvm.fabs.f16(half %sel)
    ret half %abs2
}

; Function Attrs: nocallback nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nosync nounwind speculatable willreturn memory(none) }

--- tgt ---
; ModuleID = '<stdin>'
source_filename = ''<stdin>'

define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = call half @llvm.fabs.f16(half %x)
    %cmp = fcmp ogt half %abs1, 0xH0000
    %sel = select i1 %cmp, half %x, half 0xH0000
    ret half %sel
}

; Function Attrs: nocallback nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nosync nounwind speculatable willreturn memory(none) }

--- log ---
define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = fabs half noundef %x
    %cmp = fcmp ogt half %abs1, 0x0000
    %sel = select i1 %cmp, half noundef %x, half 0x0000
    %abs2 = fabs half %sel
    ret half %abs2
}

```

```

}
=>

define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = fabs half noundef %x
    %cmp = fcmp ogt half %abs1, 0x0000
    %sel = select i1 %cmp, half noundef %x, half 0x0000
    ret half %sel
}

Transformation doesn't verify!

ERROR: Value mismatch
Example:
half noundef %x = #x8001 (-0.000000059604?)
Source:
half %abs1 = #x0001 (0.000000059604?)
i1 %cmp = #x1 (1)
half %sel = #x8001 (-0.000000059604?)
half %abs2 = #x0001 (0.000000059604?)
Target:
half %abs1 = #x0001 (0.000000059604?)
i1 %cmp = #x1 (1)
half %sel = #x8001 (-0.000000059604?)
Source value: #x0001 (0.000000059604?)
Target value: #x8001 (-0.000000059604?)
Summary:
    0 correct transformations
    1 incorrect transformations
    0 failed-to-prove transformations
    0 Alive2 errors

--- opt_stderr ---
<emph>
...

## Initial Observations ##

The reproducer is already minimized. Based on my experience in debugging these bugs, the issue is likely
introduced by the *first transformation*, or due to incorrect analysis information feeding into it. Here,
the *first transformation* means the *first transformation point* that modifies the IR in the pass
regardless of whether it's in the `instsimplify`
module or other modules.

#### Opt Information ####

To gather more insight, I ran `opt` to inspect both the transformations and the analysis results. Below
are the command and the resulting log:

```bash
$ bin/opt --passes=instsimplify -S --debug-only=instsimplify /tmp/reprod_hr__pejg.ll
; ModuleID = '/tmp/reprod_hr__pejg.ll'
source_filename = '/tmp/reprod_hr__pejg.ll'

define half @fabs_select_fabs(half noundef %x) {
entry:
 %abs1 = call half @llvm.fabs.f16(half %x)
 %cmp = fcmp ogt half %abs1, 0xH0000
 %sel = select i1 %cmp, half %x, half 0xH0000
 ret half %sel
}

; Function Attrs: nocallback nofree nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nofree nosync nounwind speculatable willreturn memory(none) }
...

From this log, I identified the first transformation point as:
...

```



```
llvm/lib/Transforms/Scalar/InstSimplifyPass.cpp::runImpl()
```

```

Here is the relevant backtrace at the moment of the transformation:

```
```stack
(frame 1) llvm/lib/Transforms/Scalar/InstSimplifyPass.cpp:56 in runImpl
(frame 2) llvm/lib/Transforms/Scalar/InstSimplifyPass.cpp:130 in llvm::InstSimplifyPass::run
```
```

Debugger Information

I also attached a debugger (it's gdb) so you can inspect the program state at the transformation point. For convenience, I started the debugger and paused execution at that point.

To use debugger, call the tool `debug` or `eval`. Call `eval` to evaluate expressions in the context of the paused frame. Call `debug` to execute debugger commands. Note, you should call `eval` as long as it suffices, as it provides a more controlled interface.

Instructions for Analysis

Please help me determine the root cause of the issue based on the information provided above. You may use the logs supplied, and you may request or infer any additional context you find necessary using the available tools.

Once you identify the issue and the corresponding fix, use the `stop` tool to specify the *edit point(s)* along with detailed reasoning. Follow this structure:

1. ****Understanding the Issue****: Explain what the problem is and why it manifests.
2. ****Analyzing `opt`'s Log****: Highlight any relevant transformations, analysis results, or unexpected behavior observed in the `opt` output.
3. ****Root Cause Analysis****: Connect the observations to the underlying cause in the code.
4. ****Proposed Edit Point(s)****
 - + Each edit point should be at least 1 lines long
 - + NOTICE (on assertion failure): Assertion failures typically indicate earlier errors in execution. Assume all assertions are correct and investigate preceding code or conditions. Edit points can contain but cannot be limited to assertion statements.
5. ****Conclusion****: Summarize the fix and its expected effect.

E.2. System Prompt Example: llvm-autofix-mini's Generate Stage

You are an expert LLVM developer. I'm encountering an LLVM bug that I haven't been able to fix. Your goal is to generate a patch that fixes the LLVM bug based on the information below.

Bug Information

Reproducer

```
```llvm
define half @fabs_select_fabs(half noundef %x) {
entry:
 %abs1 = call half @llvm.fabs.f16(half %x)
 %cmp = fcmp ogt half %abs1, 0xH0000
 %sel = select i1 %cmp, half %x, half 0xH0000
 %abs2 = call half @llvm.fabs.f16(half %sel)
 ret half %abs2
}

; Function Attrs: nocallback nofree nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nofree nosync nounwind speculatable willreturn memory(none) }
```
```

Symptom

```
```
--- src ---
define half @fabs_select_fabs(half noundef %x) {
entry:
 %abs1 = call half @llvm.fabs.f16(half %x)
```

```

 %cmp = fcmp ogt half %abs1, 0xH0000
 %sel = select i1 %cmp, half %x, half 0xH0000
 %abs2 = call half @llvm.fabs.f16(half %sel)
 ret half %abs2
}

; Function Attrs: nocallback nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nosync nounwind speculatable willreturn memory(none) }

--- tgt ---
; ModuleID = '<stdin>'
source_filename = ''<stdin>'

define half @fabs_select_fabs(half noundef %x) {
entry:
 %abs1 = call half @llvm.fabs.f16(half %x)
 %cmp = fcmp ogt half %abs1, 0xH0000
 %sel = select i1 %cmp, half %x, half 0xH0000
 ret half %sel
}

; Function Attrs: nocallback nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nosync nounwind speculatable willreturn memory(none) }

--- log ---
define half @fabs_select_fabs(half noundef %x) {
entry:
 %abs1 = fabs half noundef %x
 %cmp = fcmp ogt half %abs1, 0x0000
 %sel = select i1 %cmp, half noundef %x, half 0x0000
 %abs2 = fabs half %sel
 ret half %abs2
}
=>
define half @fabs_select_fabs(half noundef %x) {
entry:
 %abs1 = fabs half noundef %x
 %cmp = fcmp ogt half %abs1, 0x0000
 %sel = select i1 %cmp, half noundef %x, half 0x0000
 ret half %sel
}
Transformation doesn't verify!

ERROR: Value mismatch

Example:
half noundef %x = #x8001 (-0.000000059604?)

Source:
half %abs1 = #x0001 (0.000000059604?)
i1 %cmp = #x1 (1)
half %sel = #x8001 (-0.000000059604?)
half %abs2 = #x0001 (0.000000059604?)

Target:
half %abs1 = #x0001 (0.000000059604?)
i1 %cmp = #x1 (1)
half %sel = #x8001 (-0.000000059604?)
Source value: #x0001 (0.000000059604?)
Target value: #x8001 (-0.000000059604?)

Summary:
0 correct transformations
1 incorrect transformations
0 failed-to-prove transformations
0 Alive2 errors

--- opt_stderr ---

```

```
<empty>
...

Root Cause Analysis

----- BEGIN -----
1. Understanding the Issue

The bug manifests when simplifying `fabs(select(cond, x, 0))`. The transformation incorrectly removes the
outer `fabs`, replacing it with just `select(cond, x, 0)`. This is wrong because when `x` is negative and
the condition is true, the select returns the negative value, but `fabs` should make it positive.

In the test case:
- `%abs1 = fabs(%x)` where `%x = -0.000000059604` (negative denormal)
- `%cmp = (%abs1 > 0)` which is true
- `%sel = select(%cmp, %x, 0)` returns `%x` = -0.000000059604 (negative)
- `%abs2 = fabs(%sel)` should return $+0.000000059604$

But the optimization transforms this to just return `%sel` (the negative value), which is incorrect.

2. Analyzing `opt`'s Log

The opt log shows that `instsimplify` removes the %abs2 = call half @llvm.fabs.f16(half %sel)
instruction and directly returns `%sel`. The Alive2 verification correctly identifies this as an
incorrect transformation with the counterexample showing a negative denormal input.

3. Root Cause Analysis

The root cause is in `llvm/lib/Analysis/InstructionSimplify.cpp` at lines 6306-6309:

...cpp
case Intrinsic::fabs:
 if (computeKnownFPSignBit(Op0, Q) == false)
 return Op0;
 break;
...

This code attempts to simplify `fabs(Op0)` to `Op0` when `computeKnownFPSignBit` returns `false`, meaning
the sign bit is known to be zero (i.e., the value is non-negative).

However, for the select instruction `select(cond, x, 0)` where:
- True branch: `x` (can be negative)
- False branch: `0xH0000` (positive zero)

The function `computeKnownFPSignBit` is incorrectly returning `false` for this select. This happens
because when analyzing the select:
- The false branch (positive zero) has SignBit = false
- The true branch (`x`) has unknown sign
- But somewhere in the analysis, it's concluding the overall sign bit is false

Looking at the `computeKnownFPClass` code for selects (lines 4837-4887 in ValueTracking.cpp), I can see
that it correctly uses `Known |= Known2` to merge the two branches, which should set `SignBit` to
`std::nullopt` when they disagree. However, there might be additional filtering logic that's causing the
issue.

Actually, upon deeper inspection, I notice that the select handling includes filtering based on conditions
(lines 4869-4875). The condition `%cmp = fcmp ogt half %abs1, 0xH0000` tests if `fabs(x) > 0`. When this
is true, we select `x`; when false, we select `0`.

The issue is that knowing `fabs(x) > 0` doesn't tell us that `x >= 0`; it only tells us that `x != 0`.
The value `x` could still be negative!

The real bug is that `computeKnownFPSignBit` is being used incorrectly in the simplification of `fabs`.
The function returns `std::optional<bool>` where:
- `false` means sign bit is definitely 0 (value is non-negative)
- `true` means sign bit is definitely 1 (value is non-positive)
- `std::nullopt` means unknown

For the select to be safe to optimize `fabs(select(...))` to `select(...)` , we need to know that BOTH
branches of the select are non-negative. But the current code only checks if the sign bit of the result is
known to be zero, which is being incorrectly computed.
```

#### ## 4. Proposed Edit Point(s)

The edit should be at lines 6306-6309 in `llvm/lib/Analysis/InstructionSimplify.cpp`. The current code:

```
```cpp
case Intrinsic::fabs:
    if (computeKnownFPSignBit(Op0, Q) == false)
        return Op0;
    break;
```
```

This check is correct in principle - if we know the sign bit is zero (non-negative), then `fabs` is a no-op. However, the issue is that `computeKnownFPSignBit` may be incorrectly reporting `false` for select instructions in certain cases.

After further analysis, I believe the actual issue is that we should check if the value is known to be non-negative using FP class information, not just the sign bit. The sign bit check doesn't account for NaN values properly, and more importantly, the select analysis might have bugs.

A safer approach is to use `computeKnownFPClass` and check if the value `cannotBeOrderedLessThanZero()` which properly handles the 'ordered less than zero' semantics:

```
```cpp
case Intrinsic::fabs:
    if (computeKnownFPSignBit(Op0, Q) == false)
        return Op0;
    break;
```
```

Should be changed to check that the value cannot have any negative classes:

```
```cpp
case Intrinsic::fabs: {
    KnownFPClass Known = computeKnownFPClass(Op0, fcAllFlags, Q, /*Depth=*/0);
    if (Known.isKnownNever(fcNegative))
        return Op0;
    break;
}
```
```

This properly checks that all negative classes (negative zero, negative subnormal, negative normal, negative infinity) are ruled out, which is the correct condition for `fabs(x) == x`.

#### ## 5. Conclusion

The bug is in the `fabs` simplification logic in `InstructionSimplify.cpp`. The code uses `computeKnownFPSignBit` to check if a value is non-negative, but this is insufficient. The fix is to use `computeKnownFPClass` with `isKnownNever(fcNegative)` to properly check that the value cannot be negative before removing the `fabs` operation.

----- END -----

#### ## Suspected Code Locations ##

Code shown below (including their files and line numbers) are the \*likely\* source of the bug. However, the actual fix may involve other parts of the codebase as well. You should use the provided tools to explore the codebase for more context and edit the codebase as needed to fix the bug.

```
----- BEGIN -----
```cpp
// llvm/lib/Analysis/InstructionSimplify.cpp:6306-6309
6301     return Op0;
6302 }
6303
6304 Value *X;
6305 switch (IID) {
6306 case Intrinsic::fabs:
6307     if (computeKnownFPSignBit(Op0, Q) == false)
6308         return Op0;
6309     break;
6310 case Intrinsic::bswap:
6311     // bswap(bswap(x)) -> x
6312     if (match(Op0, m_BSwap(m_Value(X))))
```

```

6313     return X;
6314     break;
...
----- END -----

```

Instructions for Fixing

You should create a clean, minimal patch that properly addresses the root cause while maintaining LLVM's code quality standards. Follow these steps:

1. **Analyze**: Analyze the provided information to fully understand the bug's cause and effect. You may use provided tools to explore the codebase for more context.
2. **Propose a Fix**: Outline your proposed solution. Explain your reasoning and the specific changes you intend to make.
3. **Implement**: Use the `edit` tool to apply your proposed changes to the code.
4. **Verify**: Use the `test` tool to confirm that your patch:
 - + Is syntactically correct and does not introduce new syntax errors.
 - + Is valid and does not modify any assertions in the code.
 - + Fixes the original issue shown in the reproducer.
 - + Does not introduce any new regressions.
5. **Iterate**: If verification fails, analyze the failure, revise your proposal, and repeat the implementation and verification steps until all tests pass.
6. **Submit**: Provide the final, clean patch for review.

E.3. System Prompt Example: mini-SWE-agent

You are an expert LLVM developer. Please solve this LLVM issue:

```

----- BEGIN ISSUE -----
Type: miscompilation

```

```

Reproducer (LLVM IR): ```bash
cat /tmp/test.ll
define half @fabs_select_fabs(half noundef %x) {
entry:
  %abs1 = call half @llvm.fabs.f16(half %x)
  %cmp = fcmp ogt half %abs1, 0xH0000
  %sel = select i1 %cmp, half %x, half 0xH0000
  %abs2 = call half @llvm.fabs.f16(half %sel)
  ret half %abs2
}

```

```

; Function Attrs: nocallback nofree nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nofree nosync nounwind speculatable willreturn memory(none) }
```

```

```

LLVM's Symptom: ```bash
/llvm-autofix/build/llvm-build/152824/bin/opt /tmp/test.ll -passes=instsimplify -S

```

```

--- src ---
define half @fabs_select_fabs(half noundef %x) {
entry:
 %abs1 = call half @llvm.fabs.f16(half %x)
 %cmp = fcmp ogt half %abs1, 0xH0000
 %sel = select i1 %cmp, half %x, half 0xH0000
 %abs2 = call half @llvm.fabs.f16(half %sel)
 ret half %abs2
}

```

```

; Function Attrs: nocallback nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nosync nounwind speculatable willreturn memory(none) }

```

```

--- tgt ---
; ModuleID = '<stdin>'
source_filename = '<stdin>'

```

```

define half @fabs_select_fabs(half noundef %x) {
entry:
 %abs1 = call half @llvm.fabs.f16(half %x)

```



```

 %cmp = fcmp ogt half %abs1, 0xH0000
 %sel = select i1 %cmp, half %x, half 0xH0000
 ret half %sel
}

; Function Attrs: nocallback nosync nounwind speculatable willreturn memory(none) declare half
@llvm.fabs.f16(half) #0
attributes #0 = { nocallback nosync nounwind speculatable willreturn memory(none) }

--- log ---
define half @fabs_select_fabs(half noundef %x) {
entry:
 %abs1 = fabs half noundef %x
 %cmp = fcmp ogt half %abs1, 0x0000
 %sel = select i1 %cmp, half noundef %x, half 0x0000
 %abs2 = fabs half %sel
 ret half %abs2
}
=>
define half @fabs_select_fabs(half noundef %x) {
entry:
 %abs1 = fabs half noundef %x
 %cmp = fcmp ogt half %abs1, 0x0000
 %sel = select i1 %cmp, half noundef %x, half 0x0000
 ret half %sel
}
Transformation doesn't verify!

ERROR: Value mismatch

Example:
half noundef %x = #x8001 (-0.000000059604?)

Source:
half %abs1 = #x0001 (0.000000059604?)
i1 %cmp = #x1 (1)
half %sel = #x8001 (-0.000000059604?)
half %abs2 = #x0001 (0.000000059604?)

Target:
half %abs1 = #x0001 (0.000000059604?)
i1 %cmp = #x1 (1)
half %sel = #x8001 (-0.000000059604?)
Source value: #x0001 (0.000000059604?)
Target value: #x8001 (-0.000000059604?)

Summary:
 0 correct transformations
 1 incorrect transformations
 0 failed-to-prove transformations
 0 Alive2 errors

--- opt_stderr ---
<empty>
...
----- END ISSUE -----

```

You can execute bash commands and edit files to implement the necessary changes.

## ## Recommended Workflow

This workflows should be done step-by-step so that you can iterate on your changes and any possible problems.

1. Analyze the codebase by finding and reading relevant files
2. Edit the source code to resolve the issue
3. Verify your fix works by running the `submit-patch` command
4. Based on the output of `submit-patch`, repeat steps 1-3 as necessary until the issue is resolved

## ## Important Rules

1. Every response must contain exactly one action

2. The action must be enclosed in triple backticks
3. Directory or environment variable changes are not persistent. Every action is executed in a new subshell. However, you can prefix any action with `MY\_ENV\_VAR=MY\_VALUE cd /path/to/working/dir && ...` or write/load environment variables from files
4. Solving this issue does not require deleting any files or accessing the internet, so you are not permitted to use tools such as: rm, curl, wget, git, ssh, scp, ftp, telnet, ping, traceroute, nslookup, dig, nmap, apt, apt-get, yum, dnf, pacman, brew.

```
<system_information>
Linux 5.15.0-143-generic #153-Ubuntu SMP Fri Jun 13 19:10:45 UTC 2025 x86_64
</system_information>
```

## Formatting your response

Here is an example of a correct response:

```
<example_response>
THOUGHT: I need to understand the structure of the repository first. Let me check what files are in the
current directory to get a better understanding of the codebase.
```

```
```bash
ls -la
```
```

```
</example_response>
```

## Useful command examples

### Create a new file:

```
```bash
cat <<'EOF' > newfile.py
import numpy as np
hello = 'world'
print(hello)
EOF
```
```

### Edit files with sed:

```
```bash
# Replace all occurrences
sed -i 's/old_string/new_string/g' filename.py

# Replace only first occurrence
sed -i 's/old_string/new_string/' filename.py

# Replace first occurrence on line 1
sed -i '1s/old_string/new_string/' filename.py

# Replace all occurrences in lines 1-10
sed -i '1,10s/old_string/new_string/g' filename.py
```
```

### View file content:

```
```bash
# View specific lines with numbers
nl -ba filename.py | sed -n '10,20p'
```
```

### Submit your patch:

```
```bash
# This command has no arguments
submit-patch
```
```

### Any other permitted command you want to run:

```
```bash
anything
```
```